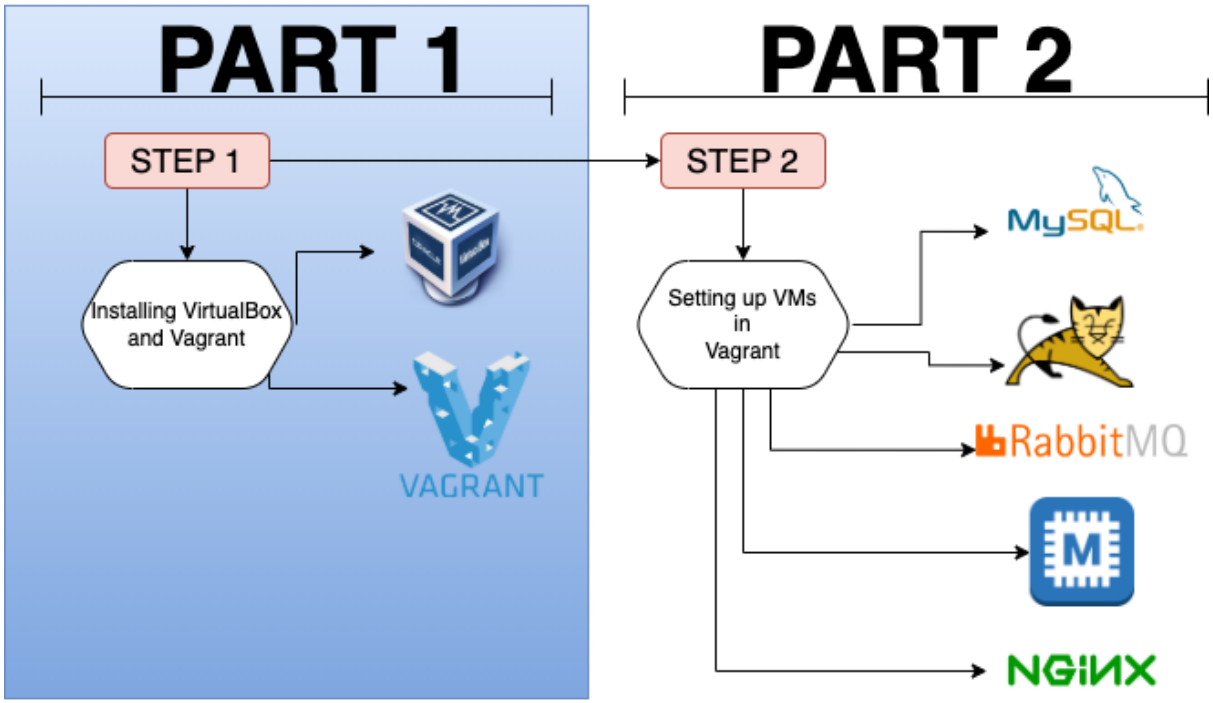
Complete DevOps Project: Multi-Tier Application Deployment Locally



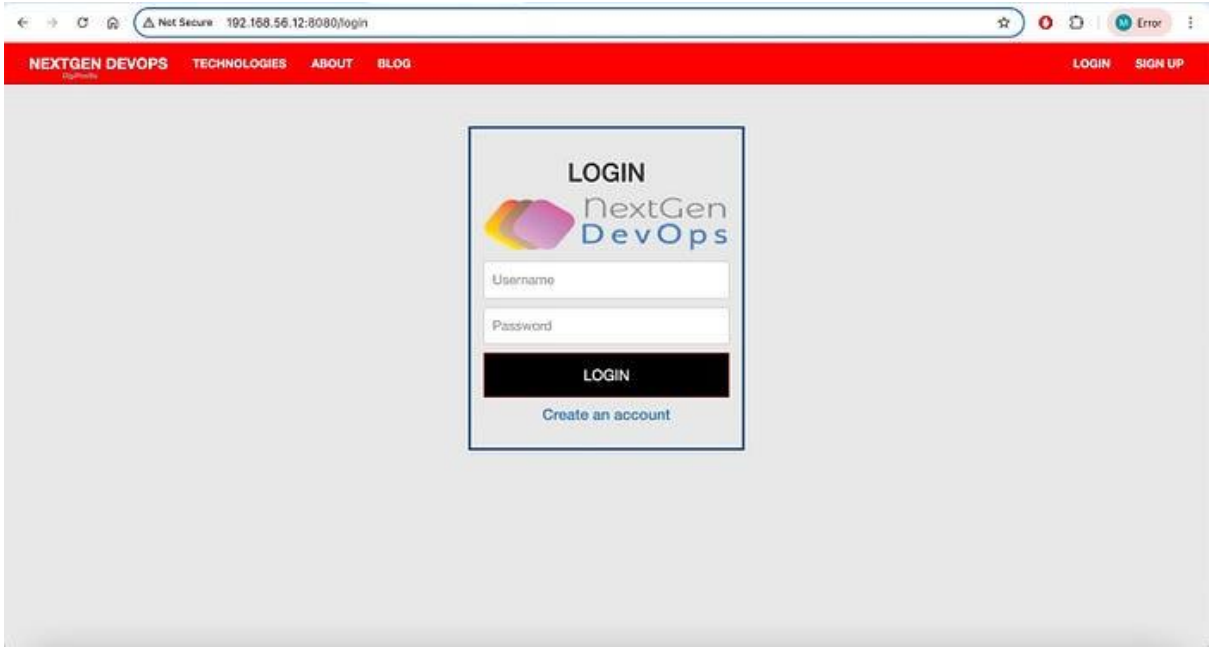
Your First Complete DevOps Project: Multi-Tier Deployment from Scratch (PART 1/2)

If you want to become a job-ready DevOps engineer, I strongly recommend not skipping any steps. It may seem challenging at first, but as you progress, you'll gain expertise.

Keep in mind that all these projects are based on real-world scenarios and are highly relevant in the industry.

Development and DevOps go hand in hand - developers build applications, while DevOps engineers ensure seamless operation across environments.

From experience, understanding the final output before deployment is crucial. Knowing its appearance, dependencies, and functionality helps in efficient setup. DevOps engineers orchestrate services to optimize performance, cost, and reliability. Once deployed, the application interface appears as follows. This Java-based application relies on five key services: **MySQL, Memcache, RabbitMQ, Tomcat, and Nginx**—working together for smooth operation across environments.



Application UI after a successful deployment

This DevOps project is detailed and extensive, so I'm breaking it into two parts. In this first part, we'll focus on setting up a local environment using **VirtualBox** and **Vagrant**.

VirtualBox is used to create virtual machines (VMs), while **Vagrant** enables the automation and management of multiple VMs through code. The first time I created a VM using Vagrant, I was truly amazed—and I'm sure you will be too! These tools are essential for mastering DevOps, and this guide will walk you through their setup and usage.

PART 1/2

Here I will show the setup in Mac environment for the setup but you use any environment to setup the same

I am writing the whole project flow and setup step by step so that you can keep track easily and focus only one step at a time. But remember for starting each step, you need to successfully complete previous step.

Install VirtualBox & Vagrant This is a single-line command on Mac. If you encounter any installation issues, feel free to ask in the comments. Sometimes, even simple tasks can take longer than expected, but every problem has a solution.

If you're new to **VirtualBox**, take some time to understand how it works. Once you're familiar with it, **Vagrant** makes managing multiple VMs incredibly easy. In this post, you'll get hands-on experience with both!

Install by brew, the VirtualBox & Vagrant

Copy> brew install virtualbox vagrant

Install vagrant plugin vagrant-hostmanager

Copy> vagrant plugin install vagrant-hostmanager

Verify that VirtualBox & Vagrant are installed. If you see different version numbers, that are fine.

Copy> VBoxManage --version

> vagrant --version

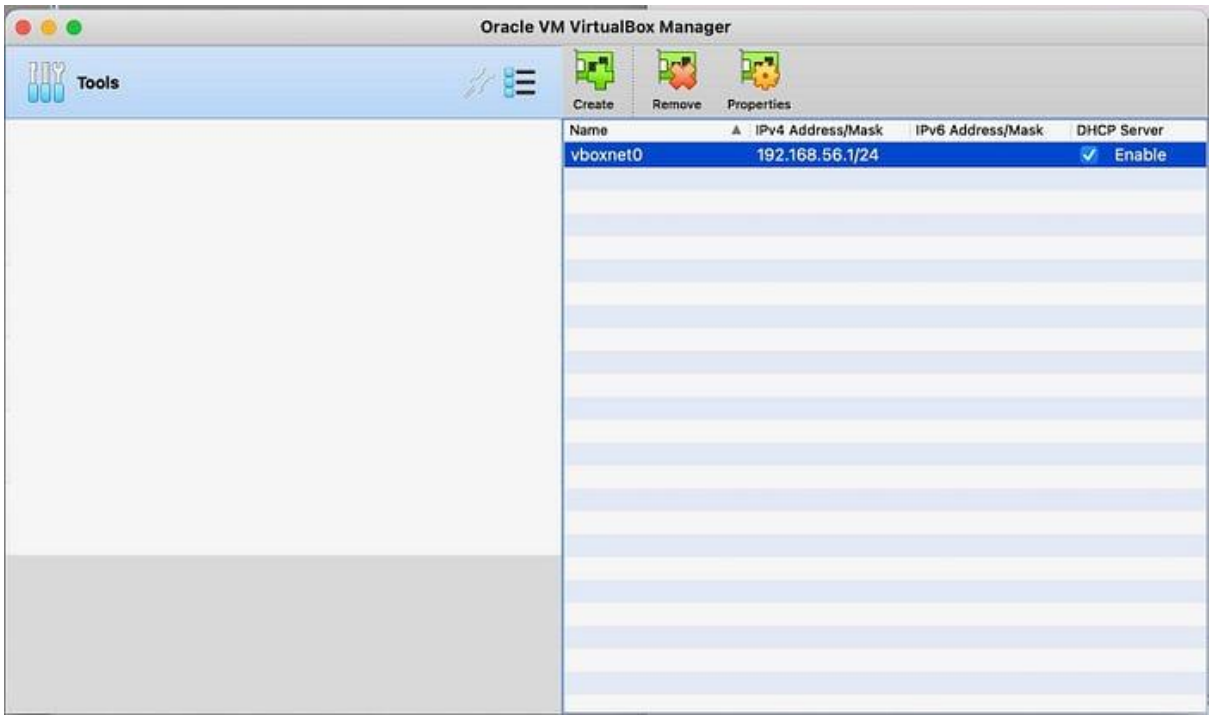
```
munir@dev ~ % VBoxManage --version
6.1.48r159471
munir@dev ~ % vagrant --version
Vagrant 2.4.1
```

Verify that the plugin vagrant-hostmanager

```
munir@dev Manual_provisioning_MacOSM1 % vagrant plugin list
vagrant-hostmanager (1.8.10, global)
```

Note: This plugin updates the **/etc/hosts** file across all newly created VMs. After setting up our five VMs in part 2, we'll verify its functionality. If you're unfamiliar with **/etc/hosts**, don't worry — we'll be using this term frequently and will provide a visual representation for clarity.

At this stage, when you open the VirtualBox, it will look like this. Your IP could be different, that's completely fine -



Screenshot of the VirtualBox interface after installation.

Up & Running 5 VMs for 5 services

Understanding Virtual Machines (VMs) in a DevOps Environment

Think of each Virtual Machine (VM) as a separate physical computer. Just like a laptop or desktop has its own **operating system (OS)** and **IP address**, each VM operates independently.

For example, if you have two computers at home connected to the same router, each gets a unique **local IP address**, allowing them to communicate. Similarly, in this setup, we will create **five VMs**—each with its own OS and IP. Services installed on these VMs will communicate with one another using their IP addresses or hostname.

When I say VMs "talk to each other," I mean that the **services running on them** communicate with each other. For instance, **Nginx** (running on one VM) might need to communicate with **Tomcat** (running on another VM). This communication happens over the network using their respective IPs.

I'm explaining this assuming you're a beginner because understanding **how services interact behind the scenes** is crucial for a DevOps engineer. When I started learning DevOps, I struggled with these concepts, so I hope this real-world analogy helps you grasp them more quickly.

Next, we'll clone a Git repository containing the **Vagrantfile**, which is essential for provisioning the five virtual machines (VMs) needed for this first part. In **Part 2**, we'll configure these VMs to set up the required services.

To clarify, each VM corresponds to one of the five key services: **MySQL, Memcache, RabbitMQ, Tomcat, and Nginx.**

Clone the git repo

Copy> git clone https://github.com/Muncodex/digiprofile-project.git

Switch to the local-vagrant branch, as it is dedicated to this project, covering both Part 1 and Part 2.

Copy> git checkout local-vagrant

```
munir@dev Code % git clone https://github.com/Muncodex/digiprofile-project.git
munir@dev Code % cd digiprofile-project
munir@dev digiprofile-project % git checkout local-vagrant
```

```
munir@dev digiprofile-project % git branch
* local-vagrant
  main
```

VM creation by Vagrant explained

As mentioned earlier, **Vagrant** works alongside **VirtualBox**, allowing you to create virtual machines through code configuration. This approach not only simplifies VM management but also enhances understanding of how environments are provisioned. Since we need **five services** running in **five separate VMs**, we'll define the Vagrant configuration to automate their creation. Let's go step by step, writing and understanding the code as we go. VM for MySQL

```
config.vm.define "db01" do |db01|
  db01.vm.box = "eurolinux-vagrant/centos-stream-9"
  db01.vm.hostname = "db01"
  db01.vm.network "private_network", ip: "192.168.56.15"
  db01.vm.provider "virtualbox" do |vb|
    vb.memory = "600"
  end
end
```

Vagrantfile configuration for the MySQL virtual machine

VM for Memcache

```
config.vm.define "mc01" do |mc01|
  mc01.vm.box = "eurolinux-vagrant/centos-stream-9"
  mc01.vm.hostname = "mc01"
  mc01.vm.network "private_network", ip: "192.168.56.14"
  mc01.vm.provider "virtualbox" do |vb|
    vb.memory = "600"
  end
end
```

Vagrantfile configuration for the Memcache virtual machine

VM for RabbitMQ

```
config.vm.define "rmq01" do |rmq01|
  rmq01.vm.box = "eurolinux-vagrant/centos-stream-9"
  rmq01.vm.hostname = "rmq01"
  rmq01.vm.network "private_network", ip: "192.168.56.13"
  rmq01.vm.provider "virtualbox" do |vb|
    vb.memory = "600"
  end
end
```

Vagrantfile configuration for the RabbitMQ virtual machine

VM for Tomcat

```
config.vm.define "app01" do |app01|
  app01.vm.box = "eurolinux-vagrant/centos-stream-9"
  app01.vm.hostname = "app01"
  app01.vm.network "private_network", ip: "192.168.56.12"
  app01.vm.provider "virtualbox" do |vb|
   vb.memory = "800"
  end
end
```

Vagrantfile configuration for the Tomcat virtual machine

VM for Nginx

```
config.vm.define "web01" do |web01|
  web01.vm.box = "ubuntu/jammy64"
  web01.vm.hostname = "web01"
  web01.vm.network "private_network", ip: "192.168.56.11"
  web01.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "800"
  end
end
```

Vagrantfile configuration for the Nginx virtual machine

Take note of the IP address and hostname defined in the Vagrant configuration for the VMs. This is essential for understanding how services communicate with each other.

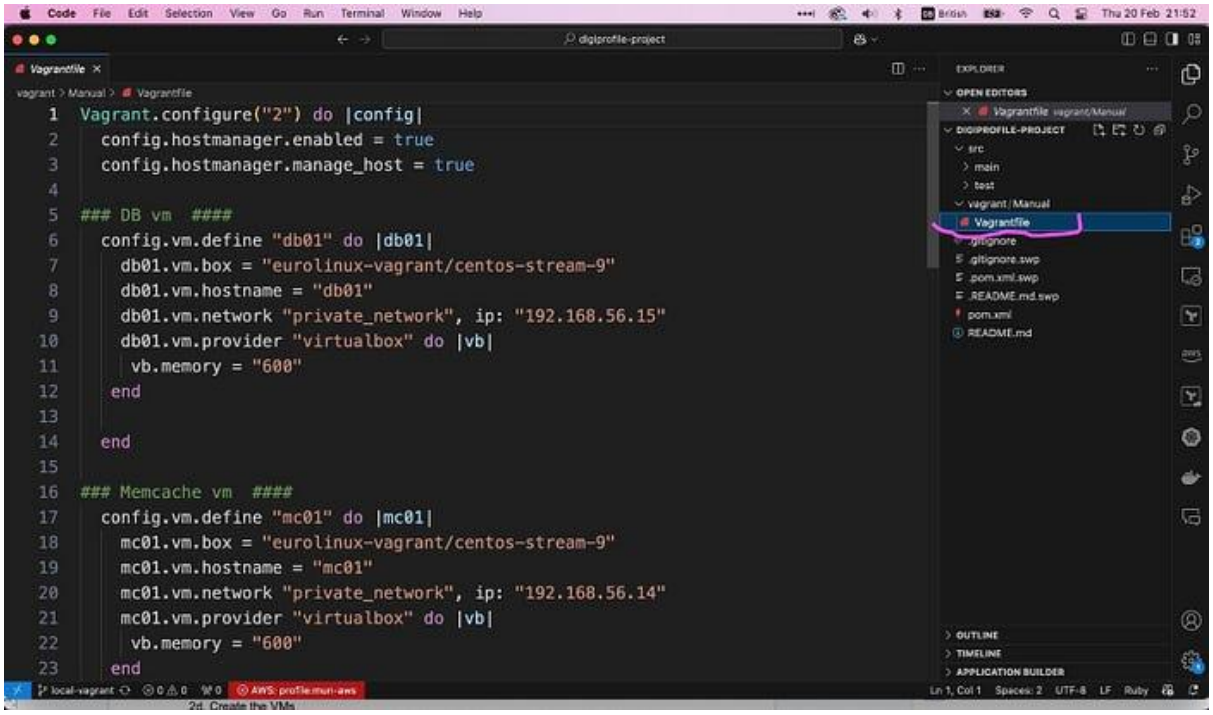
MySQL => hostname: db01, ip: 192.168.56.15 **Memcache =>** hostname: mc01, ip: 192.168.56.14 **RabbitMQ =>** hostname: rmq01, ip: 192.168.56.13 **Tomcat =>** hostname: app01, ip: 192.168.56.12 **Nginx =>** hostname: web01, ip: 192.168.56.11

Vagrantfile

Open the Vagrantfile in an IDE to go through the code.

As we cloned the repo. You can open the code in any IDE, I will show you in the Visual Studio Code but you can use any IDE you like.

Vagrant executes code in a filename Vagrantfile. So, the 5 VMs creation code in a single file name Vagrantfile and it is located in the project code location /digiprofile-project/vagrant/Manual_provisioning_MacOSM1. All the code for creating the 5 VMs written above is in one file named Vagrantfile in this location as shown on the screenshot.



Vagrantfile code and its location when opened in an IDE

Full Vagrantfile code

CopyVagrant.configure("2") do |config|

```
  config.hostmanager.enabled = true
  config.hostmanager.manage_host = true


### DB vm  ####
  config.vm.define "db01" do |db01|
    db01.vm.box = "eurolinux-vagrant/centos-stream-9"
    db01.vm.hostname = "db01"
    db01.vm.network "private_network", ip: "192.168.56.15"
    db01.vm.provider "virtualbox" do |vb|
     vb.memory = "600"
   end
  end


### Memcache vm  ####
  config.vm.define "mc01" do |mc01|
    mc01.vm.box = "eurolinux-vagrant/centos-stream-9"
```

```
    mc01.vm.hostname = "mc01"
    mc01.vm.network "private_network", ip: "192.168.56.14"
    mc01.vm.provider "virtualbox" do |vb|
     vb.memory = "600"
   end
  end

### RabbitMQ vm  ####
  config.vm.define "rmq01" do |rmq01|
    rmq01.vm.box = "eurolinux-vagrant/centos-stream-9"
  rmq01.vm.hostname = "rmq01"
    rmq01.vm.network "private_network", ip: "192.168.56.13"
    rmq01.vm.provider "virtualbox" do |vb|
     vb.memory = "600"
   end
  end

### tomcat vm ###
   config.vm.define "app01" do |app01|
    app01.vm.box = "eurolinux-vagrant/centos-stream-9"
    app01.vm.hostname = "app01"
    app01.vm.network "private_network", ip: "192.168.56.12"
    app01.vm.provider "virtualbox" do |vb|
     vb.memory = "800"
   end
   end


### Nginx VM ###
  config.vm.define "web01" do |web01|
    web01.vm.box = "ubuntu/jammy64"
```

```
    web01.vm.hostname = "web01"
  web01.vm.network "private_network", ip: "192.168.56.11"
  web01.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "800"
  end
end


end
```

Create the VMs

From the terminal, go to this location as the Vagrantfile is located in it.

Copy> cd digiprofile-project/vagrant/Manual/

```
munir@dev Code % pwd
/Users/munir/Code
munir@dev Code % cd digiprofile-project/vagrant/Manual/
munir@dev Manual % pwd
/Users/munir/Code/digiprofile-project/vagrant/Manual
munir@dev Manual % ls
Vagrantfile
```

Once you're in the directory containing the **Vagrantfile**, run the following command to create the VMs as defined in the configuration.

Verify you are on the right location

```
munir@dev Manual % pwd
/Users/munir/Code/digiprofile-project/vagrant/Manual
```

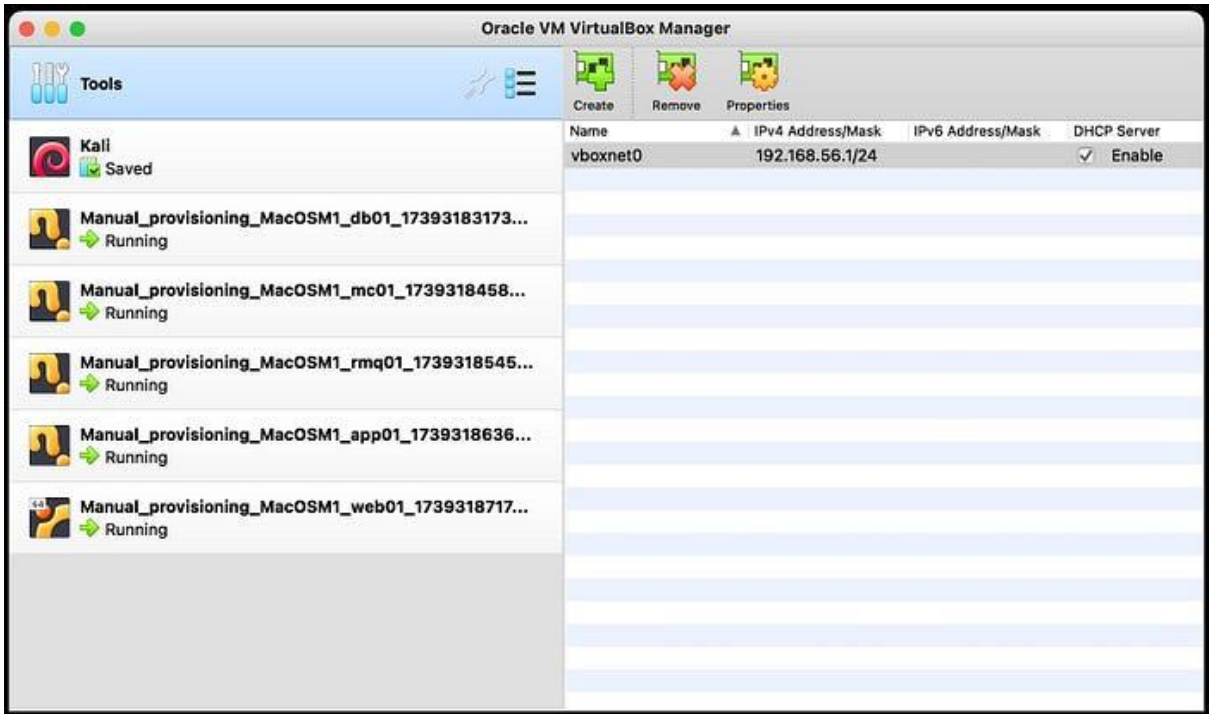
Start and provision a virtual machines (VM)

Copy> vagrant up

As shown in the screenshot, you'll be prompted to enter your computer password.

```
    db01: the guest additions and repackage the box to continue.
    db01:
    db01: This is not an error message; everything may continue to work properly,
    db01: in which case you may ignore this message.
==> db01: Setting hostname...
==> db01: Configuring and enabling network interfaces...
==> db01: Rsyncing folder: /Users/munir/Code/digiprofile-project/vagrant/Manual/ => /vagrant
==> db01: [vagrant-hostmanager:guests] Updating hosts file on active guest virtual machines...
==> db01: [vagrant-hostmanager:host] Updating hosts file on your workstation (password may be require
Password:
```

The command above will create five virtual machines (VMs). Once completed, you can open VirtualBox to see that all five VMs are now ready, as shown in the screenshot.



Each VM's name reflects the hostname defined in the **Vagrantfile**. To help you better understand the setup, I'll provide the hostname for each VM. You'll need these hostnames to facilitate communication between the VMs.

MySQL => db01 **Memcache =>** mc01 **RabbitMQ =>** rmq01 **Tomcat =>** app01 **Nginx =>** web01

Lets check the same with vagrant command

Copy> vagrant status

Access to VMs by ssh

Access to MySQL VM - hostname db01

Copy> vagrant ssh db01

Access to Memcache VM - hostname mc01

Copy> vagrant ssh mc01

Access to RabbitMQ VM - hostname rmq01

Copy> vagrant ssh rmq01

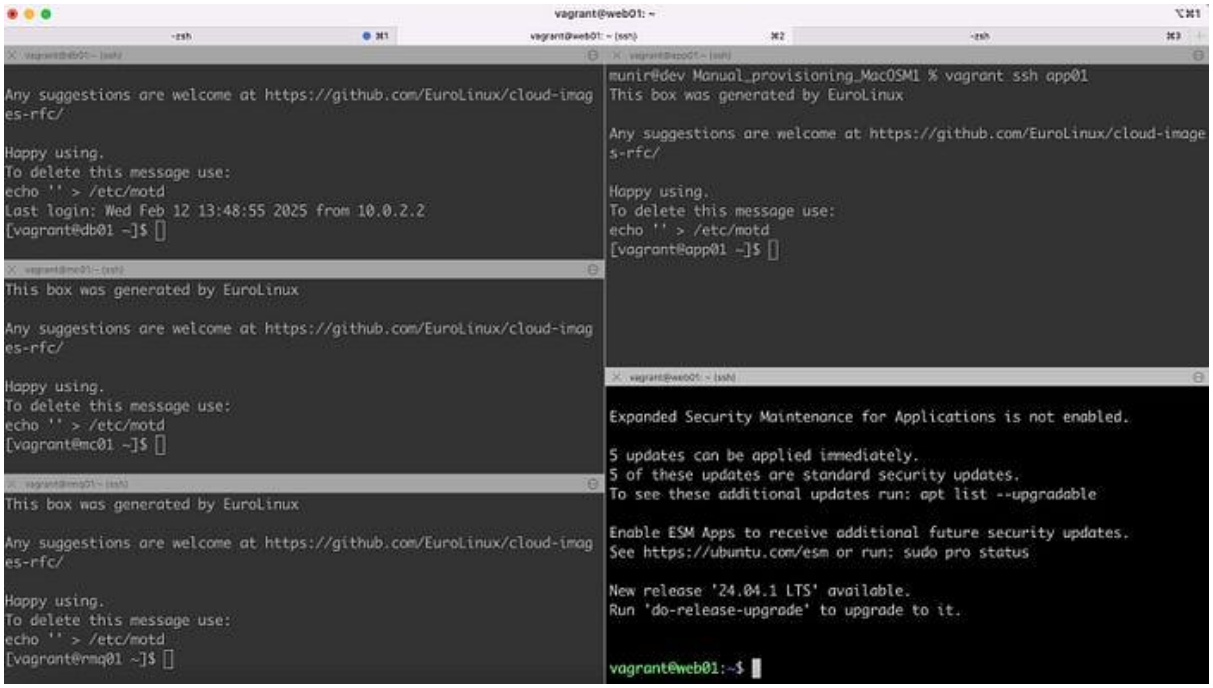
Access to Tomcat VM - hostname app01

Copy> vagrant ssh app01

Access to Nginx VM - hostname web01

Copy> vagrant ssh web01

Here is a screenshot showing how it looks after accessing all five VMs via SSH.



To split the terminal, simply use **Cmd + D** or **Shift + Cmd + D**. This allows you to work in multiple terminal panes simultaneously, making it easier to manage different tasks.

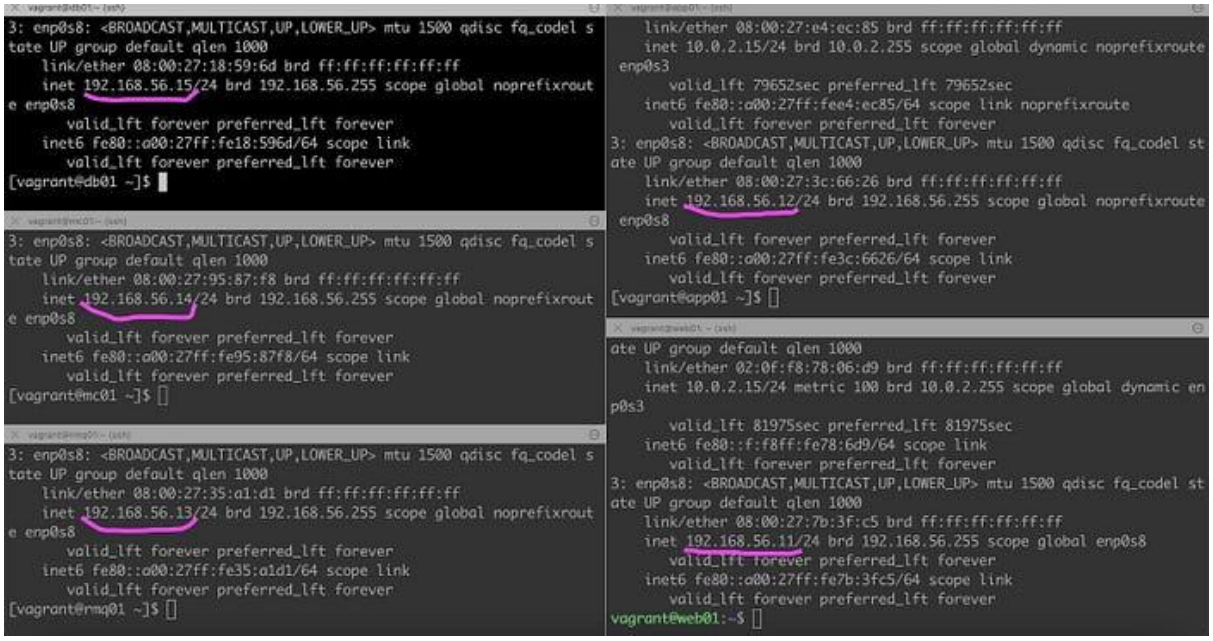
Now verify ip address of each VM. All the VMs are running in Centos Linux OS except Nginx VM which is running in Ubuntu

Check ip address in Centos by ip a

Copy> ip a

Check ip address in Ubuntu by ip addr

Copy> ip addr



Access to VMs by ssh and verify the IP address

Now we can communicate among VMs. We know the hostname and IP address of each VMs. We can communicate among VMs through their hostname or ip address

Lets verify MySQL VM to Memcache VM

Copy> vagrant ssh db01

Once inside the MySQL VM, communicated with Memcache VM by ping

Copy> ping mc01

```
[vagrant@db01 ~]$ ping mc01
PING mc01 (192.168.56.14) 56(84) bytes of data.
64 bytes from mc01 (192.168.56.14): icmp_seq=1 ttl=64 time=0.485 ms
64 bytes from mc01 (192.168.56.14): icmp_seq=2 ttl=64 time=0.453 ms
64 bytes from mc01 (192.168.56.14): icmp_seq=3 ttl=64 time=0.616 ms
^C
--- mc01 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2029ms
rtt min/avg/max/mdev = 0.453/0.518/0.616/0.070 ms
```

Similarly you can communicate with any vm by their hostname.

So our VM is up at this stage and in the next step we install the services. For example in the MySQL vm we will install MySQL, Nginx vm we will install Nginx and so on.

- Check the /etc/hosts file to view the list of VMs.
- SSH into any of the VMs to verify this.
- SSH into the Memcache VM and check the /etc/hosts file. The Memcache VM's hostname is mc01.

Copy> vagrant ssh mc01

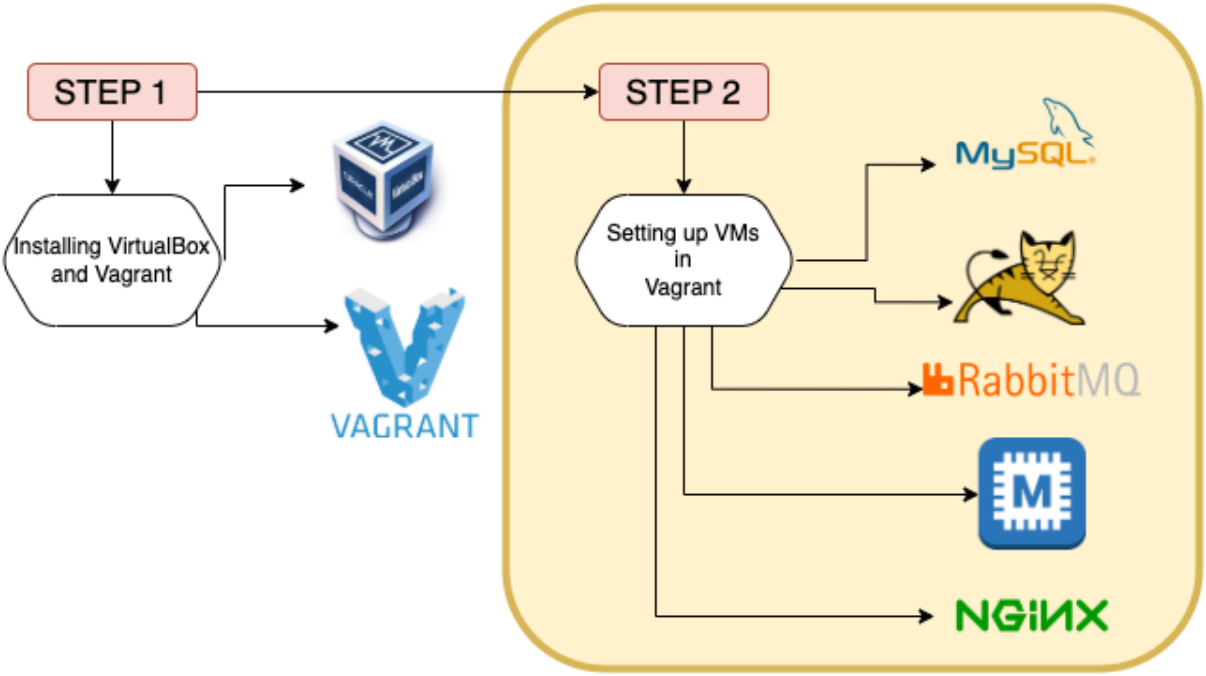
```
vagrant@mc01:~ (ssh)
munir@dev                                   % vagrant ssh mc01
This box was generated by EuroLinux

Any suggestions are welcome at https://github.com/EuroLinux/cloud-images-rfc/

Happy using.
To delete this message use:
echo '' > /etc/motd
Last login: Wed Feb 12 13:54:19 2025 from 10.0.2.2
[vagrant@mc01 ~]$ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.1.1 mc01 mc01

## vagrant-hostmanager-start
192.168.56.13   rmq01

192.168.56.11   web01

192.168.56.14   mc01

192.168.56.12   app01

192.168.56.15   db01

## vagrant-hostmanager-end
```

Congratulations! You've completed Part 1/2 of this project, where you installed VirtualBox and Vagrant in your local environment and successfully set up five VMs: MySQL, Memcache, RabbitMQ, Tomcat, and Nginx.

In the next and final part of this project (Part 2/2), we will configure these five VMs by installing the necessary services. We'll also deploy the application and provide a visual walkthrough of it running on our configured platform.

Your First Complete DevOps Project: Multi-Tier Deployment from Scratch (PART 2/2)

PART 2/2



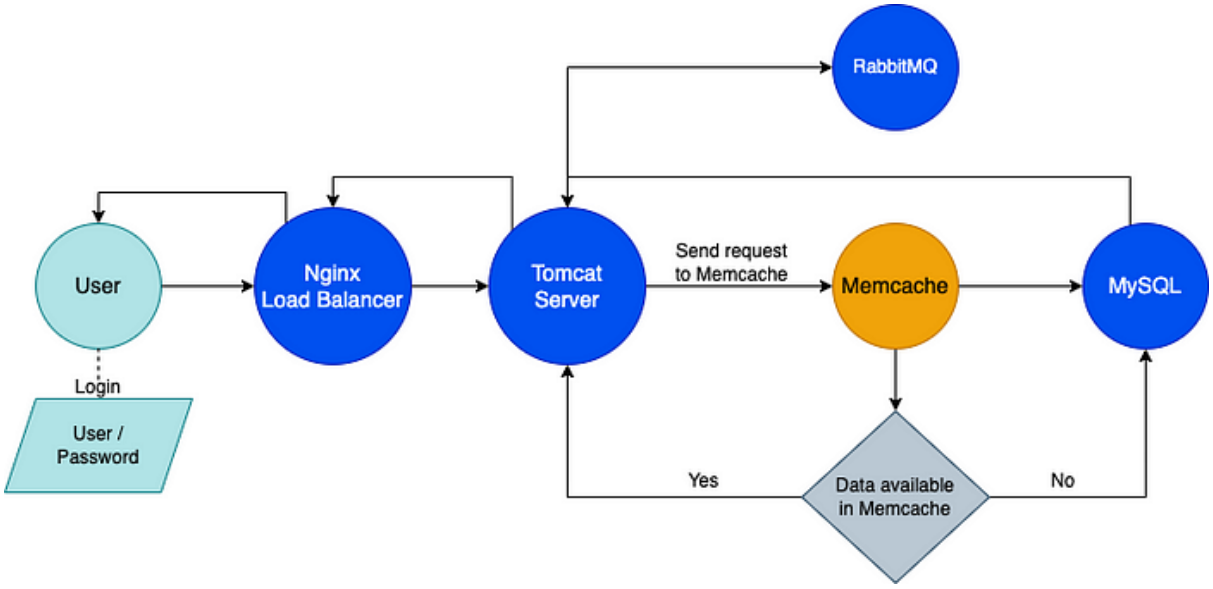
Learn to set up a multi-tier DevOps project locally using VirtualBox and Vagrant — step by step, from scratch — by configuring five VMs: MySQL, Memcache, RabbitMQ, Tomcat, and Nginx.

PART 2/2

This is continuation of part 1/2 and final part of the project, the part 2/2. In the first part, we successfully initiated five VMs for the services of MySQL, Memcache, RabbitMQ, Tomcat, and Nginx. And in this final part, we will configure each of the VMs installing relevant services and deploy the application digiprofile-project, from start to finish, cloning the repo, build, compile and deployment and finally we will verify the from the terminal and visually by the browser.

Before start configuring the VMs, lets understand the flow of the application, interactions of services and introduction of five of the services.

DevOps Project Flow



This DevOps Project Flow

This DevOps project is designed to handle web application traffic efficiently using a combination of NGINX, Tomcat, MySQL, Memcached, and RabbitMQ.

User Access & Load Balancing

- A user accesses the application through a web-based login page.
- The request first reaches **NGINX**, which acts as a load balancer and forwards it to an available **Tomcat server** for processing.

Application Processing

- **Tomcat**, a Java web application server, receives the request and processes the login attempt.
- It first checks **Memcached**, an in-memory caching system, to see if the user's session or credentials are already stored.
- If the data is not found in Memcached, Tomcat queries **MySQL**, the primary database, to authenticate the user.
- The login details are either fetched from the database or retrieved from Memcached for faster response times.

Message Queuing & Asynchronous Processing

- After handling authentication, **Tomcat sends a message to RabbitMQ**, a message broker used for asynchronous communication between services.
- RabbitMQ queues tasks like logging user activity, sending login notifications, or triggering background jobs.
- A worker service (RabbitMQ consumer) picks up these messages and processes them accordingly.

Why This Architecture?

This setup ensures: ✅ **Scalability** — NGINX efficiently distributes traffic across multiple Tomcat servers. ✅ **Performance Optimization** — Memcached reduces database load by caching frequently accessed data. ✅ **Reliable Communication** — RabbitMQ enables smooth handling of asynchronous tasks, improving system efficiency.

By integrating these components, the system achieves **high availability, optimised load management, and structured communication**, making it suitable for real-world DevOps applications. 🚀

Services Introduction

1. Nginx => Nginx is a high-performance web server and reverse proxy used for serving websites, load balancing, and handling HTTP requests efficiently.

2. Tomcat => Tomcat is an open-source Java Servlet container that runs Java web applications and serves JSP and servlet-based content.

3. RabbitMQ => RabbitMQ is an open-source message broker that facilitates communication between distributed systems by handling, storing, and forwarding messages between producers and consumers.

4. Memcache => Memcache is an in-memory key-value store used to speed up dynamic web applications by caching frequently accessed data, reducing database load and improving performance.

5. MySQL => MySQL is an open-source relational database management system (RDBMS) that uses SQL (Structured Query Language) for storing, managing, and retrieving data in a structured format.

1. Setup MySQL. VM hostname db01

ssh to db01

Copy> vagrant ssh db01

```
munir@dev Manual_provisioning_MacOSM1 % vagrant ssh db01
This box was generated by EuroLinux

Any suggestions are welcome at https://github.com/EuroLinux/cloud-images-rfc/

Happy using.
To delete this message use:
echo '' > /etc/motd
Last login: Wed Feb 12 14:11:24 2025 from 10.0.2.2
[vagrant@db01 ~]$
```

Verify hosts file content

```
[vagrant@db01 ~]$ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.1.1 db01 db01

## vagrant-hostmanager-start
192.168.56.13   rmq01

192.168.56.11   web01

192.168.56.14   mc01

192.168.56.12   app01

192.168.56.15   db01

## vagrant-hostmanager-end
```

We used CentOS for this VM and CentOS is a Linux-based operating system derived from Red Hat Enterprise Linux (RHEL).

Update CentOS

Copy> sudo yum update -y

Install MySQL / MariaDB

Copy> sudo dnf install -y mariadb-server mariadb

After installation, start and enable MariaDB to run on boot

Copy> sudo systemctl start mariadb

> sudo systemctl enable mariadb

Secure MariaDB Installation. Run the security script to set a root password as "**admin**" and remove unwanted defaults:

Copy> sudo mysql_secure_installation

```
[vagrant@db01 ~]$ sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
      SERVERS IN PRODUCTION USE!  PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] Y
Enabled successfully!
Reloading privilege tables..
 ... Success!


You already have your root account protected, so you can safely answer 'n'.

Change the root password? [Y/n] Y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
 ... Success!
```

```
By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them.  This is intended only for testing, and to make the installation
go a bit smoother.  You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
 ... Success!

Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
 ... Success!
```

Verify the MySQL access. User is **root**, password is **admin**. If you never used MySQL from terminal don't worry, keep doing, you will be familiar and expert slowly.

Copy> mysql -u root -padmin

```
[vagrant@db01 ~]$ mysql -u root -padmin
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 16
Server version: 10.5.27-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

create database accounts;

CopyMariaDB > create database accounts;

MariaDB > grant all privileges on accounts.* TO 'admin'@'localhost' identified by 'admin';

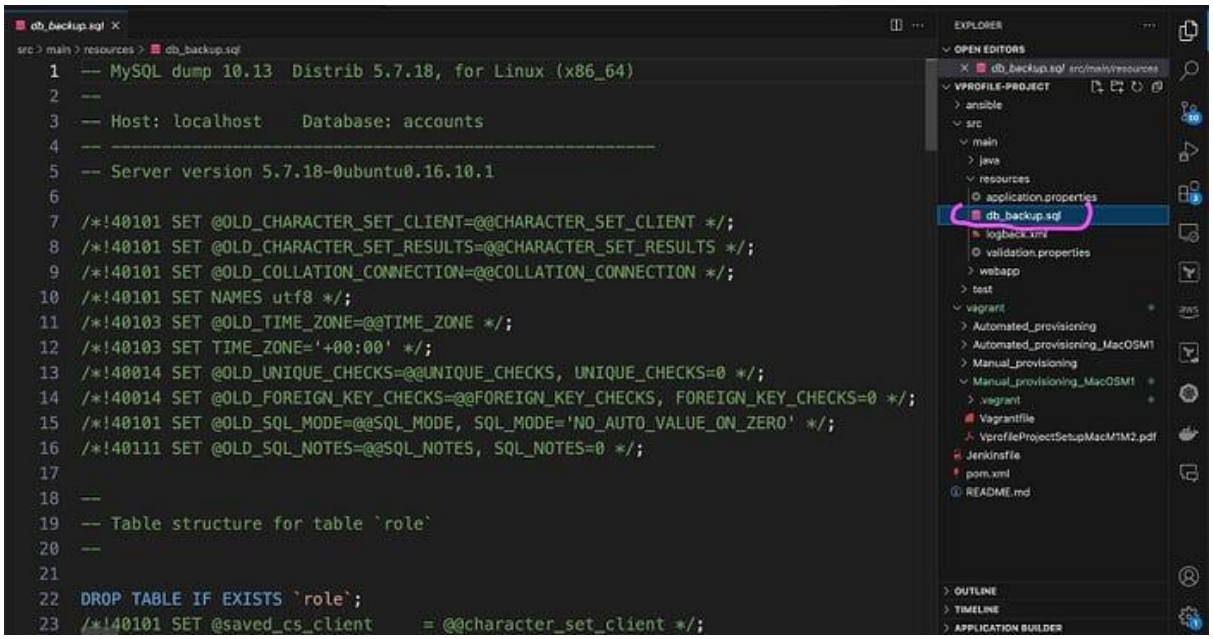
MariaDB > grant all privileges on accounts.* TO 'admin'@'%' identified by 'admin';

MariaDB > FLUSH PRIVILEGES;

MariaDB > exit;

We have a database dump file on our repo location src/main/resources/db_backup.sql. Now we will restore that dump to our created database name "accounts". Lets copy that file content to the MySQL vm db01 and restore it to the database accounts.

Open the file src/main/resources/db_backup.sql on the Visual Studio



Copy the file content -> Access to VM db01 -> create a file on the vm -> paste the file content that copied and restore that dump on the database "accounts"

User nano or vim editor whichever you are comfortable with

```
[vagrant@db01 ~]$ pwd
/home/vagrant
[vagrant@db01 ~]$ mkdir tmp
[vagrant@db01 ~]$ cd tmp/
[vagrant@db01 tmp]$ vim db_backup.sql

[3]+  Stopped                 vim db_backup.sql
[vagrant@db01 tmp]$ nano db_backup.sql
```

```
vagrant@db01:~/tmp (ssh)
  GNU nano 5.6.1                                          db_backup.sql
-- MySQL dump 10.13  Distrib 5.7.18, for Linux (x86_64)
--
-- Host: localhost    Database: accounts
-- ------------------------------------------------------
-- Server version       5.7.18-0ubuntu0.16.10.1

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Table structure for table `role`
--

DROP TABLE IF EXISTS `role`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE `role` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id`)
                                                  [ Read 121 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo     M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^_ Go To Line M-E Redo     M-6 Copy      ^Q Where Was
```

Verify the file is created

```
vagrant@db01:~/tmp (ssh)
[vagrant@db01 tmp]$ pwd
/home/vagrant/tmp
[vagrant@db01 tmp]$ ll
total 8
-rw-r--r--. 1 vagrant vagrant 6176 Feb 13 21:28 db_backup.sql
```

Restore the database dump file db_backup.sql to the database "accounts"

Copy> mysql -u root -padmin accounts < db_backup.sql

> mysql -u root -padmin accounts

```
vagrant@db01:~/tmp (ssh)
[vagrant@db01 tmp]$ pwd
/home/vagrant/tmp
[vagrant@db01 tmp]$ mysql -u root -padmin accounts < db_backup.sql
[vagrant@db01 tmp]$ mysql -u root -padmin accounts
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 27
Server version: 10.5.27-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [accounts]> show tables;
+--------------------+
| Tables_in_accounts |
+--------------------+
| role               |
| user               |
| user_role          |
+--------------------+
3 rows in set (0.001 sec)

MariaDB [accounts]>
```

Setting up firewall and allowing the mariadb to access from port 3306 because MySQL

sudo service runs at port 3306.

Copy> systemctl start firewalld

> systemctl enable firewalld

> firewall-cmd --get-active-zones

> firewall-cmd --zone=public --add-port=3306/tcp --permanent

> firewall-cmd --reload

> systemctl restart mariadb

```
[vagrant@db01 tmp]$ sudo systemctl start firewalld
[vagrant@db01 tmp]$ sudo systemctl enable firewalld
Created symlink /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service → /usr/lib/systemd/system/firewalld.service.
Created symlink /etc/systemd/system/multi-user.target.wants/firewalld.service → /usr/lib/systemd/system/firewalld.service.
[vagrant@db01 tmp]$ sudo firewall-cmd --get-active-zones
public
  interfaces: enp0s3 enp0s8
[vagrant@db01 tmp]$ sudo firewall-cmd --zone=public --add-port=3306/tcp --permanent
success
[vagrant@db01 tmp]$ sudo firewall-cmd --reload
success
[vagrant@db01 tmp]$ sudo systemctl restart mariadb
```

2. Setup Memcache. VM hostname mc01

ssh to Memcache vm

Copy> vagrant ssh to mc01

Verify hosts

```
[vagrant@mc01 ~]$ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.1.1 mc01 mc01

## vagrant-hostmanager-start
192.168.56.13   rmq01

192.168.56.11   web01

192.168.56.14   mc01

192.168.56.12   app01

192.168.56.15   db01

## vagrant-hostmanager-end
```

Update CentOS. Run the following command to update your system packages:

Copy> sudo yum update -y

```
[vagrant@mc01 ~]$ pwd
/home/vagrant
[vagrant@mc01 ~]$ sudo yum update -y
```

Install memcache, start & enable, allowing it to be accessible from any IP address, not just localhost, it on port 11211

Copy> sudo dnf install epel-release -y

> sudo dnf install memcached -y

> sudo systemctl start memcached

> sudo systemctl enable memcached

> sudo systemctl status memcached

> sed -i 's/127.0.0.1/0.0.0.0/g' /etc/sysconfig/memcached

> sudo systemctl restart memcached

Setting Up Firewall Rules for Memcached (Ports 11211 & 11111)

Copy> sudo systemctl start firewalld

> sudo systemctl enable firewalld

> sudo firewall-cmd --add-port=11211/tcp

> sudo firewall-cmd --runtime-to-permanent

> sudo firewall-cmd --add-port=11111/udp

> sudo firewall-cmd --runtime-to-permanent

> sudo memcached -p 11211 -U 11111 -u memcached -d

Verify firewall ports

Copy> sudo firewall-cmd --list-ports

```
[vagrant@mc01 ~]$ sudo firewall-cmd --list-ports
11211/tcp 11111/udp
```

Check if Memcached is listening on port 11211

```
[vagrant@mc01 ~]$ nc -zv 127.0.0.1 11211
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Connected to 127.0.0.1:11211.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
```

3. Setup RabbitMQ. VM hostname rmq01

Ssh to RabbitMQ vm

Copy> vagrant ssh rmq01

```
munir@dev Manual_provisioning_MacOSM1 % vagrant ssh rmq01
This box was generated by EuroLinux

Any suggestions are welcome at https://github.com/EuroLinux/cloud-images-rfc/

Happy using.
To delete this message use:
echo '' > /etc/motd
Last login: Wed Feb 12 13:54:29 2025 from 10.0.2.2
[vagrant@rmq01 ~]$
```

Update CentOS

Run the following command to update your system packages:

Copy> sudo yum update -y

```
[vagrant@mc01 ~]$ pwd
/home/vagrant
[vagrant@mc01 ~]$ sudo yum update -y
```

Install RabbitMQ and enable service.

Copy> sudo dnf install epel-release -y

> sudo dnf -y install centos-release-rabbitmq-38

> sudo dnf --enablerepo=centos-rabbitmq-38 -y install rabbitmq-server

> sudo systemctl enable --now rabbitmq-server

Allow external access and set up user 'test' as an administrator in RabbitMQ ✅ Allowing external access (loopback_users, []) ✅ Creating the user test ✅ Granting admin privileges to test ✅ Setting permissions ✅ Restarting RabbitMQ to apply changes

Copy> sudo echo "loopback_users = none" | sudo tee /etc/rabbitmq/rabbitmq.conf

> sudo rabbitmqctl add_user test test

> sudo rabbitmqctl set_user_tags test administrator

> sudo rabbitmqctl set_permissions -p / test ".*" ".*" ".*"

> sudo systemctl restart rabbitmq-server

Starting the Firewall and Allowing Port 5672 for RabbitMQ Access

Copy> sudo ss -tulnp | grep beam

```
[vagrant@rmq01 rabbitmq]$ sudo ss -tulnp | grep beam
tcp   LISTEN 0      128          0.0.0.0:25672      0.0.0.0:*    users:(("beam.smp",pid=46861,fd=18))
tcp   LISTEN 0      128                *:5672             *:*    users:(("beam.smp",pid=46861,fd=33))
```

4. Setup Tomcat. VM hostname app01

Ssh to Tomcat vm

Copy> vagrant ssh app01

Verify hosts

```
[vagrant@app01 ~]$ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.1.1 app01 app01

## vagrant-hostmanager-start
192.168.56.13   rmq01

192.168.56.11   web01

192.168.56.14   mc01

192.168.56.12   app01

192.168.56.15   db01

## vagrant-hostmanager-end
```

Check CentOS version

```
[vagrant@app01 ~]$ cat /etc/centos-release
CentOS Stream release 9
```

Run the following command to update your system packages:

Copy> sudo yum update -y

```
[vagrant@mc01 ~]$ pwd
/home/vagrant
[vagrant@mc01 ~]$ sudo yum update -y
```

Installing Java, Git, Maven, and Wget

Copy> sudo dnf -y install java-11-openjdk java-11-openjdk-devel

> sudo dnf install git maven wget -y

Verify Java and Maven installed version

Copy> java --version

> mvn -version

```
[vagrant@app01 ~]$ java --version
openjdk 17.0.14 2025-01-21 LTS
OpenJDK Runtime Environment (Red_Hat-17.0.14.0.7-1) (build 17.0.14+7-LTS)
OpenJDK 64-Bit Server VM (Red_Hat-17.0.14.0.7-1) (build 17.0.14+7-LTS, mixed mode, sharing)
[vagrant@app01 ~]$ mvn -version
Apache Maven 3.6.3 (Red Hat 3.6.3-22)
Maven home: /usr/share/maven
Java version: 17.0.14, vendor: Red Hat, Inc., runtime: /usr/lib/jvm/java-17-openjdk-17.0.14.0.7-2.el9.x86_64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "5.14.0-514.el9.x86_64", arch: "amd64", family: "unix"
```

Downloading Apache Tomcat 9.0.75 using Wget

Copywget https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.75/bin/apache-tomcat-9.0.75.tar.gz

```
[vagrant@app01 tmp]$ ls
apache-tomcat-9.0.75.tar.gz
```

Unzip that downloaded file

Copy> tar xzvf apache-tomcat-9.0.75.tar.gz

```
[vagrant@app01 tmp]$ ll apache-tomcat-9.0.75
total 132
drwxr-x---. 2 vagrant vagrant  4096 Feb 15 18:01 bin
-rw-r-----. 1 vagrant vagrant 19992 May  4  2023 BUILDING.txt
drwx------. 2 vagrant vagrant  4096 May  4  2023 conf
-rw-r-----. 1 vagrant vagrant  6210 May  4  2023 CONTRIBUTING.md
drwxr-x---. 2 vagrant vagrant  4096 Feb 15 18:01 lib
-rw-r-----. 1 vagrant vagrant 57092 May  4  2023 LICENSE
drwxr-x---. 2 vagrant vagrant     6 May  4  2023 logs
-rw-r-----. 1 vagrant vagrant  2333 May  4  2023 NOTICE
-rw-r-----. 1 vagrant vagrant  3398 May  4  2023 README.md
-rw-r-----. 1 vagrant vagrant  6901 May  4  2023 RELEASE-NOTES
-rw-r-----. 1 vagrant vagrant 16505 May  4  2023 RUNNING.txt
drwxr-x---. 2 vagrant vagrant    30 Feb 15 18:01 temp
drwxr-x---. 7 vagrant vagrant    81 May  4  2023 webapps
drwxr-x---. 2 vagrant vagrant     6 May  4  2023 work
```

Add user name tomcat. This is typically done when setting up Apache Tomcat, ensuring that the tomcat user exists but cannot log in, enhancing security.

Copy> sudo useradd --home-dir /usr/local/tomcat --shell /sbin/nologin tomcat

Copy Tomcat all files that extracted from zip to /usr/local/tomcat that created for tomcat user

Copy> sudo cp -r /tmp/apache-tomcat-9.0.75/* /usr/local/tomcat/

Make tomcat user owner of tomcat home dir

Copy> sudo chown -R tomcat.tomcat /usr/local/tomcat

Lets verify that folder /usr/local/tomcat owns by tomcat user

```
[vagrant@app01 tmp]$ ll /usr/local
total 4
drwxr-xr-x. 2 root   root      6 Jun 25  2024 bin
drwxr-xr-x. 2 root   root      6 Jun 25  2024 etc
drwxr-xr-x. 2 root   root      6 Jun 25  2024 games
drwxr-xr-x. 2 root   root      6 Jun 25  2024 include
drwxr-xr-x. 2 root   root      6 Jun 25  2024 lib
drwxr-xr-x. 3 root   root     17 Jun 25  2024 lib64
drwxr-xr-x. 2 root   root      6 Jun 25  2024 libexec
drwxr-xr-x. 2 root   root      6 Jun 25  2024 sbin
drwxr-xr-x. 5 root   root     49 Jun 25  2024 share
drwxr-xr-x. 2 root   root      6 Jun 25  2024 src
drwx------. 9 tomcat tomcat 4096 Feb 15 18:20 tomcat
```

Creating a Systemd Service for Tomcat

Copy> sudo vim /etc/systemd/system/tomcat.service

[Unit]

Description=Tomcat

After=network.target

[Service]

User=tomcat

WorkingDirectory=/usr/local/tomcat

Environment=JRE_HOME=/usr/lib/jvm/jre

Environment=JAVA_HOME=/usr/lib/jvm/jre

Environment=CATALINA_HOME=/usr/local/tomcat

Environment=CATALINE_BASE=/usr/local/tomcat

ExecStart=/usr/local/tomcat/bin/catalina.sh run

ExecStop=/usr/local/tomcat/bin/shutdown.sh

SyslogIdentifier=tomcat-%i

[Install]

WantedBy=multi-user.target

Lets verify

```
[vagrant@app01 tmp]$ cat /etc/systemd/system/tomcat.service
[Unit]
Description=Tomcat
After=network.target

[Service]
User=tomcat
WorkingDirectory=/usr/local/tomcat
Environment=JRE_HOME=/usr/lib/jvm/jre
Environment=JAVA_HOME=/usr/lib/jvm/jre
Environment=CATALINA_HOME=/usr/local/tomcat
Environment=CATALINE_BASE=/usr/local/tomcat
ExecStart=/usr/local/tomcat/bin/catalina.sh run
ExecStop=/usr/local/tomcat/bin/shutdown.sh
SyslogIdentifier=tomcat-%i

[Install]
WantedBy=multi-user.target
```

Reload systemd file

Copy> sudo systemctl daemon-reload

Start & Enable service

Copy> sudo systemctl start tomcat

> sudo systemctl enable tomcat

> sudo systemctl status tomcat

```
[vagrant@app01 tmp]$ sudo systemctl daemon-reload
[vagrant@app01 tmp]$ sudo systemctl start tomcat
[vagrant@app01 tmp]$ sudo systemctl enable tomcat
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat.service.
[vagrant@app01 tmp]$ sudo systemctl status tomcat
● tomcat.service - Tomcat
     Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
     Active: active (running) since Sat 2025-02-15 19:10:38 UTC; 14s ago
   Main PID: 46772 (java)
      Tasks: 30 (limit: 4480)
     Memory: 91.0M
        CPU: 4.071s
     CGroup: /system.slice/tomcat.service
             └─46772 /usr/lib/jvm/jre/bin/java -Djava.util.logging.config.file=/usr/local/tomcat/conf/logging.properties -Djava.u

Feb 15 19:10:40 app01 tomcat-[46772]: 15-Feb-2025 19:10:40.595 INFO [main] org.apache.catalina.startup.HostConfig.deployDirectory
Feb 15 19:10:40 app01 tomcat-[46772]: 15-Feb-2025 19:10:40.647 INFO [main] org.apache.catalina.startup.HostConfig.deployDirectory
Feb 15 19:10:40 app01 tomcat-[46772]: 15-Feb-2025 19:10:40.647 INFO [main] org.apache.catalina.startup.HostConfig.deployDirectory
```

Configuring Firewall to Allow Tomcat on Port 8080

Copy> sudo systemctl start firewalld

> sudo systemctl enable firewalld

> sudo firewall-cmd --get-active-zones

> sudo firewall-cmd --zone=public --add-port=8080/tcp --permanent

> sudo firewall-cmd --reload

Check if port 8080 is open

Copy> sudo firewall-cmd --zone=public --query-port=8080/tcp

```
[vagrant@app01 tmp]$ sudo firewall-cmd --zone=public --query-port=8080/tcp
yes
```

Setting Memory Limits for Maven

Copy> export MAVEN_OPTS="-Xmx512m"

Code build and deploy. Java code needs to build before deployment.

- Clone the repo
- checkout to branch: local-setup-mun
- Staying in the /tmp folder

Copy> git clone https://github.com/Muncodex/digiprofile-project.git

> cd digiprofile-project/

> git checkout local-setup-mun

Make sure you are at the branch: **local-setup-mun**

```
[vagrant@app01 tmp]$ pwd
/tmp
[vagrant@app01 tmp]$ git clone https://github.com/Muncodex/vprofile-project.git
Cloning into 'vprofile-project'...
remote: Enumerating objects: 3817, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3817 (delta 0), reused 0 (delta 0), pack-reused 3815 (from 2)
Receiving objects: 100% (3817/3817), 111.34 MiB | 22.64 MiB/s, done.
Resolving deltas: 100% (1415/1415), done.
[vagrant@app01 tmp]$ cd vprofile-project/
[vagrant@app01 vprofile-project]$ git checkout local-setup-mun
branch 'local-setup-mun' set up to track 'origin/local-setup-mun'.
Switched to a new branch 'local-setup-mun'
[vagrant@app01 vprofile-project]$ git branch
* local-setup-mun
  vp-rem
```

Check the folder and files

```
[vagrant@app01 vprofile-project]$ ll
total 16
drwxr-xr-x. 3 vagrant vagrant  108 Feb 15 22:21 ansible
-rw-r--r--. 1 vagrant vagrant 3936 Feb 15 22:21 Jenkinsfile
-rw-r--r--. 1 vagrant vagrant 7311 Feb 15 22:21 pom.xml
-rw-r--r--. 1 vagrant vagrant  517 Feb 15 22:21 README.md
drwxr-xr-x. 4 vagrant vagrant   30 Feb 15 22:21 src
drwxr-xr-x. 6 vagrant vagrant  136 Feb 15 22:21 vagrant
```

Check the files at src/main/resources/

```
[vagrant@app01 vprofile-project]$ ll src/main/resources/
total 20
-rw-r--r--. 1 vagrant vagrant  717 Feb 15 22:21 application.properties
-rw-r--r--. 1 vagrant vagrant 6176 Feb 15 22:21 db_backup.sql
-rw-r--r--. 1 vagrant vagrant  581 Feb 15 22:21 logback.xml
-rw-r--r--. 1 vagrant vagrant  280 Feb 15 22:21 validation.properties
```

We need to update the application.properties file, as it contains the application's configuration settings. The configurations should be adjusted according to our VM host, IP, username, password, and other relevant details. This file is essential for the application to connect to the backend services, as it defines all backend service endpoints and authentication credentials.

Open the file with vim or nano whichever you are comfortable with.

MySQL vm db01:

- User: root
- Pass: admin

Note: Before running any command, I always use the pwd command in the terminal to display my current location. This ensures clarity on the required directory before executing any further commands.

```
[vagrant@app01 vprofile-project]$ pwd
/tmp/vprofile-project
[vagrant@app01 vprofile-project]$ vim src/main/resources/application.properties
```

application.properties

Copy#JDBC Configutation for Database Connection

jdbc.driverClassName=com.mysql.jdbc.Driver

jdbc.url=jdbc:mysql://db01:3306/accounts?useUnicode=true&characterEncoding=UTF-8&zeroDateTimeBehavior=convertToNull

jdbc.username=admin

jdbc.password=admin

#Memcached Configuration For Active and StandBy Host

#For Active Host

memcached.active.host=mc01

memcached.active.port=11211

#For StandBy Host

memcached.standBy.host=127.0.0.2

memcached.standBy.port=11211

#RabbitMq Configuration

rabbitmq.address=rmq01

rabbitmq.port=5672

rabbitmq.username=test

rabbitmq.password=test

#Elasticesearch Configuration

elasticsearch.host =192.168.1.85

elasticsearch.port =9300

elasticsearch.cluster=digiprofile

elasticsearch.node=digiprofilenode

Match your file with this one; it should be exactly the same. Go through the code and ensure that the hostname, username, password, port, and other details match.

Now we are ready to make our source code to package by maven.

Following command will make the package from source code.

```
[vagrant@app01 vprofile-project]$ pwd
/tmp/vprofile-project
[vagrant@app01 vprofile-project]$ mvn install
```

Once the above command finishes, it creates a new directory named '**target**,' where the compiled application is saved.

```
[vagrant@app01 vprofile-project]$ pwd
/tmp/vprofile-project
[vagrant@app01 vprofile-project]$ ll
total 20
drwxr-xr-x.  3 vagrant vagrant  108 Feb 15 22:21 ansible
-rw-r--r--.  1 vagrant vagrant 3936 Feb 15 22:21 Jenkinsfile
-rw-r--r--.  1 vagrant vagrant 7311 Feb 15 22:21 pom.xml
-rw-r--r--.  1 vagrant vagrant  517 Feb 15 22:21 README.md
drwxr-xr-x.  4 vagrant vagrant   30 Feb 15 22:21 src
drwxr-xr-x. 11 vagrant vagrant 4096 Feb 16 00:28 target
drwxr-xr-x.  6 vagrant vagrant  136 Feb 15 22:21 vagrant
```

Check files on the "**target**" folder

```
[vagrant@app01 vprofile-project]$ ll target/
total 50232
drwxr-xr-x. 3 vagrant vagrant      186 Feb 16 00:28 classes
drwxr-xr-x. 3 vagrant vagrant       25 Feb 16 00:28 generated-sources
drwxr-xr-x. 3 vagrant vagrant       30 Feb 16 00:28 generated-test-sources
-rw-r--r--. 1 vagrant vagrant    83748 Feb 16 00:28 jacoco.exec
drwxr-xr-x. 2 vagrant vagrant       28 Feb 16 00:28 maven-archiver
drwxr-xr-x. 3 vagrant vagrant       35 Feb 16 00:28 maven-status
drwxr-xr-x. 3 vagrant vagrant       20 Feb 16 00:28 site
drwxr-xr-x. 2 vagrant vagrant     4096 Feb 16 00:28 surefire-reports
drwxr-xr-x. 3 vagrant vagrant       17 Feb 16 00:28 test-classes
drwxr-xr-x. 5 vagrant vagrant       54 Feb 16 00:28 vprofile-v2
-rw-r--r--. 1 vagrant vagrant 51346381 Feb 16 00:28 vprofile-v2.war
```

Lets focus on the file **digiprofile-v2.war** in this directory.

- The .war file (Web Application Archive) is used to deploy Java-based web applications on a server Tomcat.
- This file created as war because our project is configured as a WAR project in **pom.xml** file, indicated by:

Copy<packaging>war</packaging>

Why is it named digiprofile-v2.war?

The name is derived from Maven project's artifact ID and version, as defined in **pom.xml**

Copy<artifactId>digiprofile</artifactId>

<packaging>war</packaging>

<version>v2</version>

Lets explore what is an **artifact** in Maven. It is just the **final output file** that Maven creates when you build a project. Its crucial to know about artifact to be a DevOps expert.

Think of it like baking a cake:

- The **recipe** (Maven configuration in pom.xml) tells how to make it.

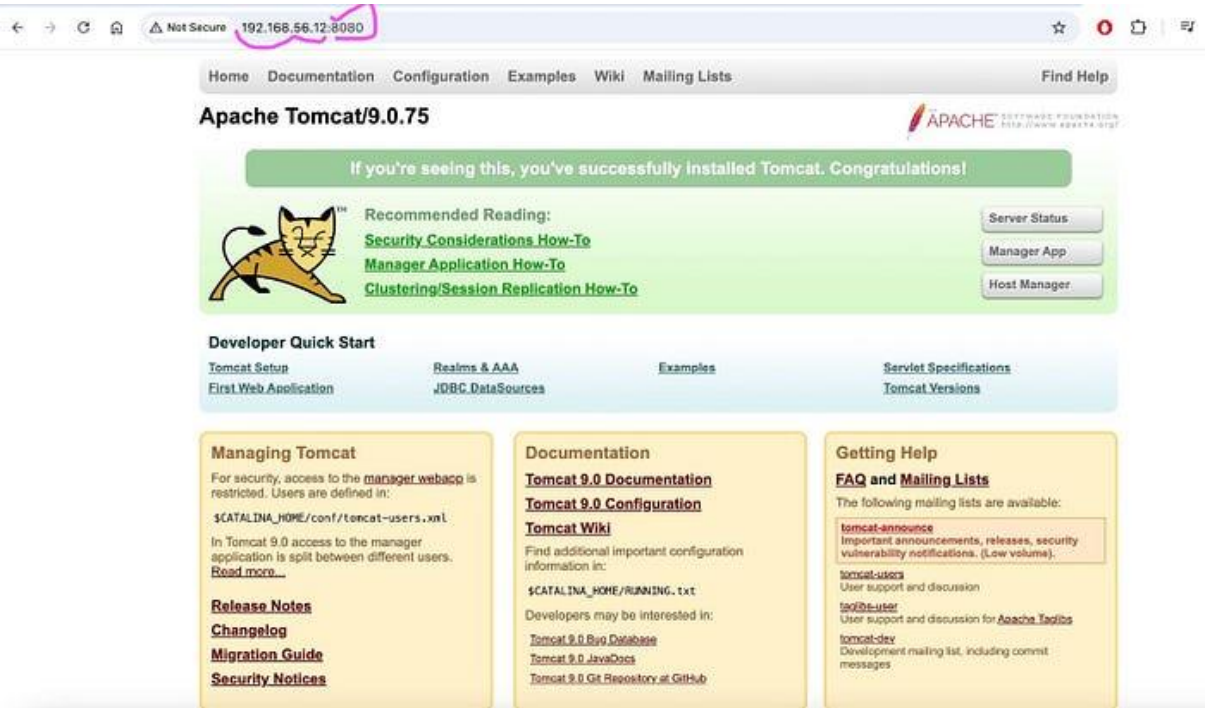
- The **ingredients** (source code and dependencies) are put together.
- The **oven** (Maven build process) bakes everything.
- The **cake** (artifact) is the final product you get at the end!

In our case, the artifact is digiprofile-v2.war, which is a **ready-to-use web application file** that can be deployed to a server Tomcat and in the next step, we will do that.

Before going to the next step, lets check the default page hosted at the tomcat server. The VM name of the Tomcat server is **app01** and the ip address is **192.168.56.12** and it is running on the **8080**. So browsing endpoint is **192.168.56.12:8080**

```
### tomcat vm ###
  config.vm.define "app01" do |app01|
   app01.vm.box = "eurolinux-vagrant/centos-stream-9"
   app01.vm.hostname = "app01"
   app01.vm.network "private_network", ip: "192.168.56.12"
   app01.vm.provider "virtualbox" do |vb|
    vb.memory = "800"
  end
  end
```

192.168.56.12:8080



Switch to the root user to avoid using sudo for every command.

Copy> sudo -i

```
[vagrant@app01 vprofile-project]$ sudo -i
[root@app01 ~]#
```

The default webpage is displayed by Tomcat when we browse to 192.168.56.12:8080. It is served from the location: /usr/local/tomcat/webapps/ROOT/.

```
[root@app01 ~]# ll /usr/local/tomcat/webapps/ROOT/
total 164
-rw-r-----. 1 tomcat tomcat 27235 Feb 15 18:20 asf-logo-wide.svg
-rw-r-----. 1 tomcat tomcat   713 Feb 15 18:20 bg-button.png
-rw-r-----. 1 tomcat tomcat  1918 Feb 15 18:20 bg-middle.png
-rw-r-----. 1 tomcat tomcat  1401 Feb 15 18:20 bg-nav.png
-rw-r-----. 1 tomcat tomcat  3103 Feb 15 18:20 bg-upper.png
-rw-r-----. 1 tomcat tomcat 21630 Feb 15 18:20 favicon.ico
-rw-r-----. 1 tomcat tomcat 12234 Feb 15 18:20 index.jsp
-rw-r-----. 1 tomcat tomcat  6901 Feb 15 18:20 RELEASE-NOTES.txt
-rw-r-----. 1 tomcat tomcat  5542 Feb 15 18:20 tomcat.css
-rw-r-----. 1 tomcat tomcat 67795 Feb 15 18:20 tomcat.svg
drwxr-x---. 2 tomcat tomcat    21 Feb 15 18:20 WEB-INF
```

To ensure our application runs correctly, we will perform the following steps:

- Remove the ROOT directory.
- Copy the digiprofile-v2.war file to this location and rename it as ROOT.war.

Remove ROOT directory

```
[root@app01 ~]# rm -rf /usr/local/tomcat/webapps/ROOT
[root@app01 ~]# ll /usr/local/tomcat/webapps
total 4
drwxr-x---. 16 tomcat tomcat 4096 Feb 15 18:20 docs
drwxr-x---.  7 tomcat tomcat   99 Feb 15 18:20 examples
drwxr-x---.  6 tomcat tomcat   79 Feb 15 18:20 host-manager
drwxr-x---.  6 tomcat tomcat  114 Feb 15 18:20 manager
```

Make sure you are at this location

```
[root@app01 vprofile-project]# pwd
/tmp/vprofile-project
```

Copy the digiprofile-v2.war file to the specified location and rename it as ROOT.war.

```
[root@app01 vprofile-project]# cp target/vprofile-v2.war /usr/local/tomcat/webapps/ROOT.war
[root@app01 vprofile-project]# ll /usr/local/tomcat/webapps
total 50148
drwxr-x---. 16 tomcat tomcat     4096 Feb 15 18:20 docs
drwxr-x---.  7 tomcat tomcat       99 Feb 15 18:20 examples
drwxr-x---.  6 tomcat tomcat       79 Feb 15 18:20 host-manager
drwxr-x---.  6 tomcat tomcat      114 Feb 15 18:20 manager
drwxr-x---.  5 tomcat tomcat       54 Feb 16 07:41 ROOT
-rw-r--r--.  1 root   root   51346381 Feb 16 07:41 ROOT.war
```

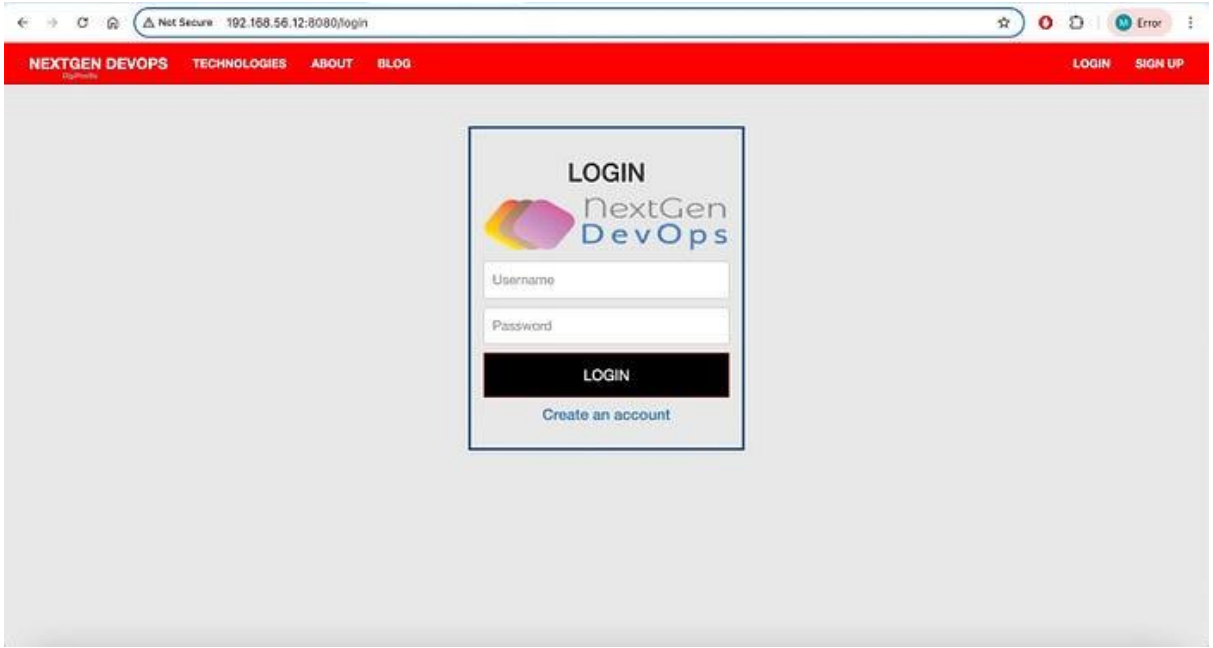
Now, let's check the endpoint to verify if our application is displayed after updating the build RAR file. At this point, you may see a blank page with a 404 — Not Found error. This occurs because the

ROOT.RAR file is owned by the root user, as shown in the screenshot above. To resolve this, we need to change the ownership of the file to the **tomcat** user. We'll walk through this step in next.

Change the owner to tomcat

Copy> chown tomcat.tomcat /usr/local/tomcat/webapps -R

Check again **192.168.56.12:8080** and application shows perfectly **User:** nextgen7 **Pass:** nextgen7



In the next step, we will set up the Nginx web server on the VM named '**web01**,' which is running inUbuntu.

5. Setup Nginx. VM hostname web01

ssh to Nginx vm

Copy> vagrant ssh web01

Switch to the root user.

Copy> sudo -i

Verify hosts

Copy> cat /etc/hosts

```
root@web01:~# cat /etc/hosts
127.0.0.1       localhost

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost   ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
127.0.1.1       ubuntu-jammy    ubuntu-jammy

127.0.2.1 web01 web01

## vagrant-hostmanager-start
192.168.56.13   rmq01

192.168.56.11   web01

192.168.56.14   mc01

192.168.56.12   app01

192.168.56.15   db01

## vagrant-hostmanager-end
```

Update & Install Nginx

Copy> apt update

> apt upgrade

> apt install nginx -y

Create a Nginx config file. Use vim or nano for it.

Copy> vim /etc/nginx/sites-available/digiprofileapp

```
upstream digiprofileapp {
  server app01:8080;
}


server {
  listen 80;
  location / {
```

```
    proxy_pass http://digiprofileapp;
  }
}
```

Remove default Nginx config file

Copy> rm -rf /etc/nginx/sites-enabled/default

The following command will create a symbolic link from the Nginx site configuration file for digiprofileapp, located in /etc/nginx/sites-available/, to the /etc/nginx/sites-enabled/ directory. The symlink allows Nginx to enable the configuration without duplicating the file, which is a common practice for managing Nginx server blocks.

Copy> ln -s /etc/nginx/sites-available/digiprofileapp

/etc/nginx/sites-enabled/digiprofileapp

Restart Nginx to apply the changes.

Copy> systemctl restart nginx

All ready at this point. Now we can access the application through nginx. Let browse through Nginx vm ip which is running in port 80